

a03_1

November 14, 2025

1 1 Intuition on SVD

Report of Jonas Ortner: 2265527 Marmee Pandya: 1963521

```
[2]: import numpy as np
      from numpy.linalg import svd as svd
      from numpy.linalg import matrix_rank as matrix_rank
      import matplotlib.pyplot as plt

      %load_ext autoreload
      %autoreload 2
      from a03_helper import *
```

```
[3]: M1 = np.array(
      [
          [1, 1, 1, 0, 0],
          [1, 1, 1, 0, 0],
          [1, 1, 1, 0, 0],
          [0, 0, 0, 0, 0],
          [0, 0, 0, 0, 0],
      ]
      )

      M2 = np.array(
      [
          [0, 0, 0, 0, 0],
          [0, 2, 1, 2, 0],
          [0, 2, 1, 2, 0],
          [0, 2, 1, 2, 0],
          [0, 0, 0, 0, 0],
      ]
      )

      M3 = np.array([[0, 0, 0, 0], [0, 1, 1, 1], [0, 1, 1, 1], [0, 1, 1, 1], [0, 1, 1, 1]])

      M4 = np.array(
      [
```

```

        [1, 1, 1, 0, 0],
        [1, 1, 1, 0, 0],
        [1, 1, 1, 0, 0],
        [0, 0, 0, 1, 1],
        [0, 0, 0, 1, 1],
    ]
)

M5 = np.array(
    [
        [1, 1, 1, 0, 0],
        [1, 1, 1, 0, 0],
        [1, 1, 1, 1, 1],
        [0, 0, 1, 1, 1],
        [0, 0, 1, 1, 1],
    ]
)

M6 = np.array(
    [
        [1, 1, 1, 1, 1],
        [1, 1, 1, 1, 1],
        [1, 1, 0, 1, 1],
        [1, 1, 1, 1, 1],
        [1, 1, 1, 1, 1],
    ]
)

```

1.1 1b

[4]: *# Your part*

```

U, s, Vt = np.linalg.svd(M1)
S = np.diag(s)
U, S, Vt

```

```

[4]: (array([[ -5.77350269e-01,  8.16496581e-01, -1.57496771e-16,
               0.00000000e+00,  0.00000000e+00],
             [ -5.77350269e-01, -4.08248290e-01, -7.07106781e-01,
               0.00000000e+00,  0.00000000e+00],
             [ -5.77350269e-01, -4.08248290e-01,  7.07106781e-01,
               0.00000000e+00,  0.00000000e+00],
             [  0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
               0.00000000e+00,  1.00000000e+00],
             [  0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
               1.00000000e+00,  0.00000000e+00]]),
      array([[3.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00,
              0.00000000e+00],

```

```

[0.00000000e+00, 2.55806258e-17, 0.00000000e+00, 0.00000000e+00,
 0.00000000e+00],
[0.00000000e+00, 0.00000000e+00, 2.11125548e-48, 0.00000000e+00,
 0.00000000e+00],
[0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00,
 0.00000000e+00],
[0.00000000e+00, 0.00000000e+00, 0.00000000e+00, 0.00000000e+00,
 0.00000000e+00]]),
array([[ -0.57735027, -0.57735027, -0.57735027, -0.          , -0.          ],
       [ 0.81649658, -0.40824829, -0.40824829,  0.          ,  0.          ],
       [ 0.          , -0.70710678,  0.70710678,  0.          ,  0.          ],
       [ 0.          ,  0.          ,  0.          ,  0.          ,  1.          ],
       [ 0.          ,  0.          ,  0.          ,  1.          ,  0.          ]]))

```

```

[5]: # Your part
U, s, Vt = np.linalg.svd(M2)
S = np.diag(s)
U, s, Vt

```

```

[5]: (array([[ 0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
               0.00000000e+00,  1.00000000e+00],
             [-5.77350269e-01,  8.16496581e-01, -2.21595527e-16,
               0.00000000e+00,  0.00000000e+00],
             [-5.77350269e-01, -4.08248290e-01, -7.07106781e-01,
               0.00000000e+00,  0.00000000e+00],
             [-5.77350269e-01, -4.08248290e-01,  7.07106781e-01,
               0.00000000e+00,  0.00000000e+00],
             [ 0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
               1.00000000e+00,  0.00000000e+00]]),
      array([5.19615242e+00, 4.67036192e-17, 3.64967471e-48, 0.00000000e+00,
             0.00000000e+00]),
      array([[ -0.          , -0.66666667, -0.33333333, -0.66666667, -0.          ],
             [ 0.          ,  0.74535599, -0.2981424 , -0.59628479,  0.          ],
             [ 0.          ,  0.          , -0.89442719,  0.4472136 ,  0.          ],
             [ 0.          ,  0.          ,  0.          ,  0.          ,  1.          ],
             [ 1.          ,  0.          ,  0.          ,  0.          ,  0.          ]]))

```

```

[6]: # Your part
U, s, Vt = np.linalg.svd(M3)
S = np.diag(s)
U, s, Vt

```

```

[6]: (array([[ 0.00000000e+00,  0.00000000e+00,  0.00000000e+00,
               1.00000000e+00,  0.00000000e+00],
             [-5.00000000e-01,  8.66025404e-01, -1.66533454e-16,
               0.00000000e+00, -4.16333634e-17],
             [-5.00000000e-01, -2.88675135e-01,  8.16496581e-01,

```

```

0.00000000e+00, -5.81681443e-17],
[-5.00000000e-01, -2.88675135e-01, -4.08248290e-01,
0.00000000e+00, -7.07106781e-01],
[-5.00000000e-01, -2.88675135e-01, -4.08248290e-01,
0.00000000e+00, 7.07106781e-01]]),
array([3.46410162e+00, 7.85046229e-17, 3.26618704e-49, 0.00000000e+00]),
array([[ -0.          , -0.57735027, -0.57735027, -0.57735027],
[ 0.          , 0.81649658, -0.40824829, -0.40824829],
[ 0.          , 0.          , -0.70710678, 0.70710678],
[ 1.          , 0.          , 0.          , 0.          ]]))

```

```

[7]: # Your part
U, s, Vt = np.linalg.svd(M4)
S = np.diag(s)
U, s, Vt

```

```

[7]: (array([[ -5.77350269e-01, 0.00000000e+00, 0.00000000e+00,
8.16496581e-01, -1.57496771e-16],
[ -5.77350269e-01, 0.00000000e+00, 0.00000000e+00,
-4.08248290e-01, -7.07106781e-01],
[ -5.77350269e-01, 0.00000000e+00, 0.00000000e+00,
-4.08248290e-01, 7.07106781e-01],
[ 0.00000000e+00, -7.07106781e-01, -7.07106781e-01,
0.00000000e+00, 0.00000000e+00],
[ 0.00000000e+00, -7.07106781e-01, 7.07106781e-01,
0.00000000e+00, 0.00000000e+00]]),
array([3.00000000e+00, 2.00000000e+00, 3.35470445e-17, 2.55806258e-17,
2.11125548e-48]),
array([[ -0.57735027, -0.57735027, -0.57735027, -0.          , -0.          ],
[ -0.          , -0.          , -0.          , -0.70710678, -0.70710678],
[ -0.          , -0.          , -0.          , 0.70710678, -0.70710678],
[ 0.81649658, -0.40824829, -0.40824829, 0.          , 0.          ],
[ 0.          , -0.70710678, 0.70710678, 0.          , 0.          ]]))

```

```

[ ]: # Your part
U, s, Vt = np.linalg.svd(M5)
S = np.diag(s)
U, s, Vt

```

```

[ ]: (array([[ -3.94102719e-01, -5.00000000e-01, 3.07706105e-01,
7.07106781e-01, -7.78284580e-17],
[ -3.94102719e-01, -5.00000000e-01, 3.07706105e-01,
-7.07106781e-01, 8.90009193e-17],
[ -6.15412209e-01, -1.38652485e-16, -7.88205438e-01,
3.70081756e-18, -1.11724613e-17],
[ -3.94102719e-01, 5.00000000e-01, 3.07706105e-01,
1.31421212e-17, -7.07106781e-01],

```

```

        [-3.94102719e-01,  5.00000000e-01,  3.07706105e-01,
         1.31421212e-17,  7.07106781e-01]]),
array([3.56155281e+00, 2.00000000e+00, 5.61552813e-01, 4.91793136e-17,
       3.69964287e-49]),
array([[ -3.94102719e-01, -3.94102719e-01, -6.15412209e-01,
        -3.94102719e-01, -3.94102719e-01],
       [-5.00000000e-01, -5.00000000e-01, -2.03023610e-16,
        5.00000000e-01,  5.00000000e-01],
       [-3.07706105e-01, -3.07706105e-01,  7.88205438e-01,
        -3.07706105e-01, -3.07706105e-01],
       [ 7.07106781e-01, -7.07106781e-01, -1.99101353e-16,
        4.46551847e-17,  4.46551847e-17],
       [ 0.00000000e+00, -1.59738305e-18, -3.74617773e-17,
        -7.07106781e-01,  7.07106781e-01]]))

```

```

[12]: # Your part
      U, s, Vt = np.linalg.svd(M6)
      S = np.diag(s)
      U, s, Vt

```

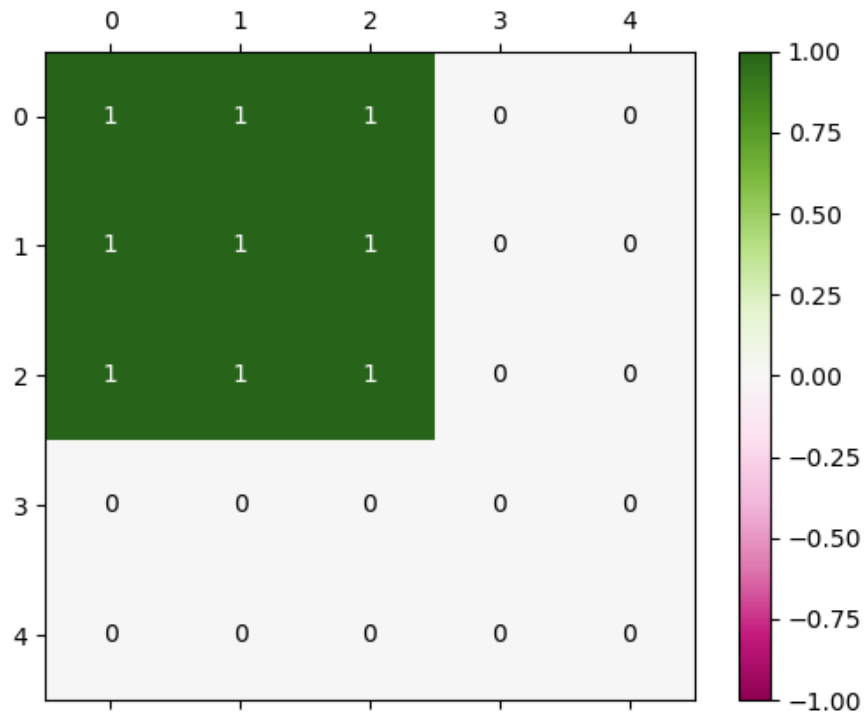
```

[12]: (array([[ -4.61939766e-01, -1.91341716e-01,  8.66024213e-01,
         1.43585580e-03, -6.53979252e-17],
       [ -4.61939766e-01, -1.91341716e-01, -2.90028476e-01,
         8.16016840e-01, -2.28495382e-16],
       [ -3.82683432e-01,  9.23879533e-01,  2.31316397e-19,
         4.33453405e-17, -8.63142633e-18],
       [ -4.61939766e-01, -1.91341716e-01, -2.87997869e-01,
        -4.08726348e-01, -7.07106781e-01],
       [ -4.61939766e-01, -1.91341716e-01, -2.87997869e-01,
        -4.08726348e-01,  7.07106781e-01]]),
array([4.82842712e+00, 8.28427125e-01, 9.95090019e-17, 2.18529703e-17,
       5.31822283e-50]),
array([[ -4.61939766e-01, -4.61939766e-01, -3.82683432e-01,
        -4.61939766e-01, -4.61939766e-01],
       [ 1.91341716e-01,  1.91341716e-01, -9.23879533e-01,
         1.91341716e-01,  1.91341716e-01],
       [ 8.47659026e-01, -4.49816411e-01,  4.08702351e-17,
        -1.98921308e-01, -1.98921308e-01],
       [ 1.77409629e-01,  7.40044051e-01, -3.17602784e-17,
        -4.58726840e-01, -4.58726840e-01],
       [ 0.00000000e+00, -8.19361931e-17,  7.96038953e-19,
        -7.07106781e-01,  7.07106781e-01]]))

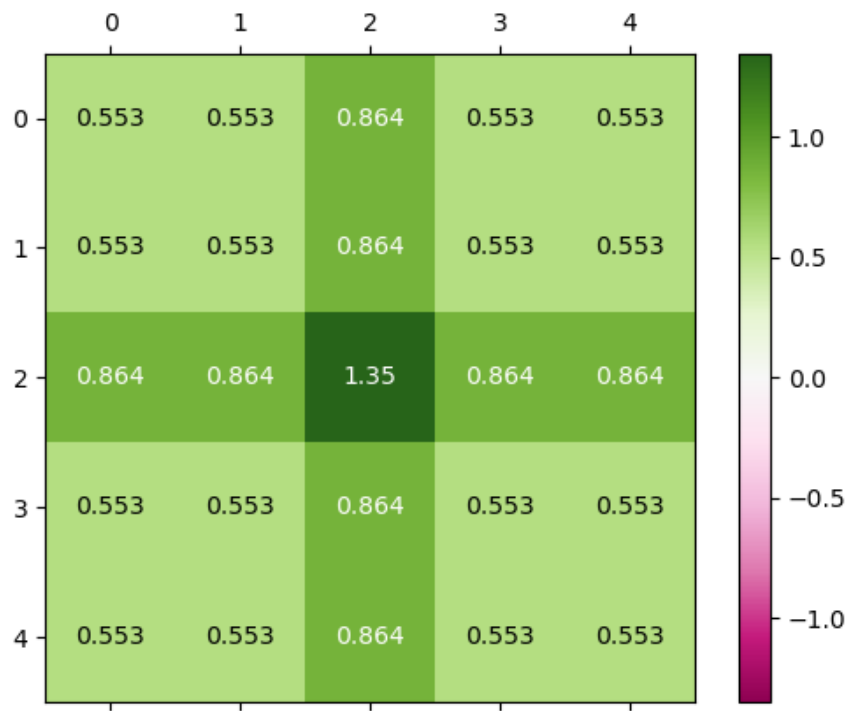
```

1.2 1c

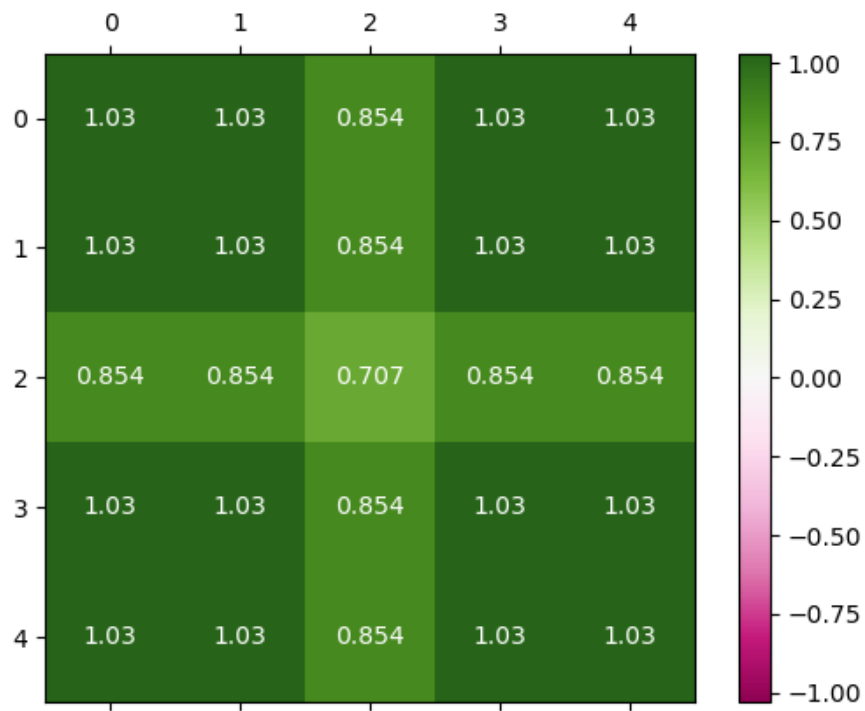
```
[9]: # You can use the functions svdcomp and plot_matrix from a03_helper.py to
      ↪ visualize the results.
      plot_matrix(svdcomp(M4, range(1)))
```



```
[10]: # You can use the functions svdcomp and plot_matrix from a03_helper.py to
       ↪ visualize the results.
       plot_matrix(svdcomp(M5, range(1)))
```



```
[11]: # You can use the functions svdcomp and plot_matrix from a03_helper.py to
      ↪ visualize the results.
      plot_matrix(svdcomp(M6, range(1)))
```



1.3 1d

```
[ ]: # Another method to compute the rank is matrix_rank.  
     # YOUR PART
```


a03_2

November 14, 2025

1 2 The SVD on Weather Data

Report of Jonas Ortner: 2265527 Marmee Pandya: 1963521

```
[ ]: import numpy as np
      from numpy.linalg import svd as svd
      from numpy.linalg import matrix_rank as matrix_rank
      import pandas as pd
      import matplotlib.pyplot as plt

      %load_ext autoreload
      %autoreload 2
      from a03_helper import *
```

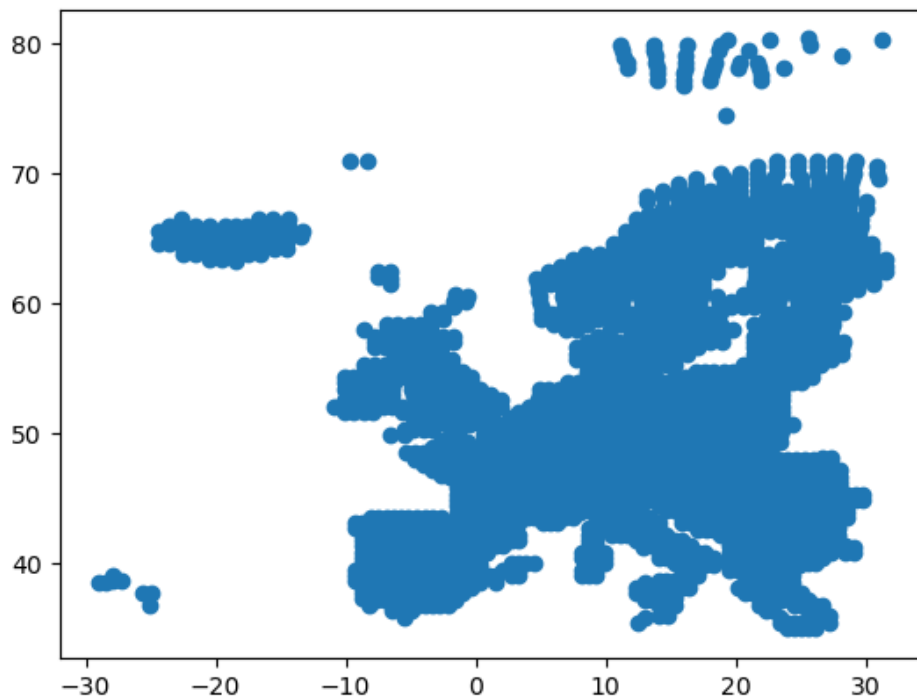
The autoreload extension is already loaded. To reload it, use:

```
%reload_ext autoreload
```

```
[ ]: # The data is being loaded via `a03_helper.py`
      climate = pd.read_csv("data/worldclim.csv")
      coord = pd.read_csv("data/worldclim_coordinates.csv")

      lon = coord["lon"]
      lat = coord["lat"]

      # Plot the coordinates
      plot_xy(lon, lat)
```



1.1 2a

```
[ ]: # YOUR PART
# Normalize the data to z-scores. Store the result in X.
# For this, complete the code in `a03_helper.py`.
X = (climate - climate.mean()) / climate.std()

print(f'Each feature circa mean = 0: {(X.mean().abs() < 1e-6).all()}')
print(f'Each feature circa standard deviation = 1:{(X.std().sub(1).abs() < 1e-6).all()}')
```

Each feature circa mean = 0: True
Each feature circa standard deviation = 1:True

```
[ ]: # Plot histograms of attributes
nextplot()
X.hist(ax=plt.gca())
```

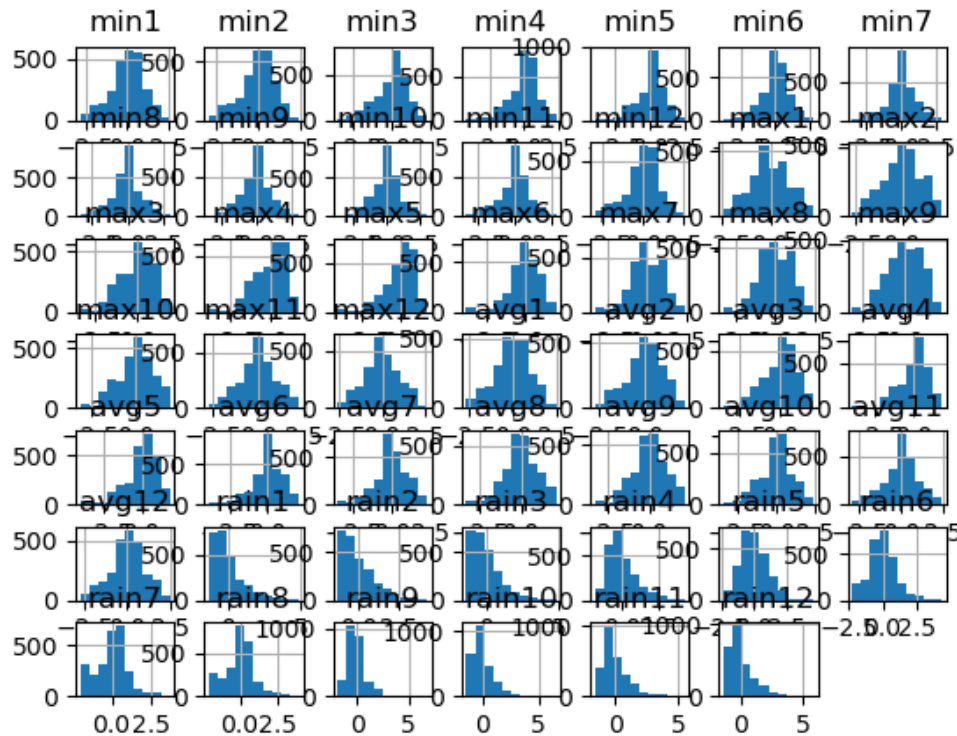
/var/folders/z5/pr2ddd2n7118kplp8zz0fklh0000gn/T/ipykernel_4931/2722728386.py:3:
UserWarning: To output multiple subplots, the figure containing the passed axes
is being cleared.
X.hist(ax=plt.gca())

```

[ ]: array([[<Axes: title={'center': 'min1'}>,
             <Axes: title={'center': 'min2'}>,
             <Axes: title={'center': 'min3'}>,
             <Axes: title={'center': 'min4'}>,
             <Axes: title={'center': 'min5'}>,
             <Axes: title={'center': 'min6'}>,
             <Axes: title={'center': 'min7'}>],
            [<Axes: title={'center': 'min8'}>,
             <Axes: title={'center': 'min9'}>,
             <Axes: title={'center': 'min10'}>,
             <Axes: title={'center': 'min11'}>,
             <Axes: title={'center': 'min12'}>,
             <Axes: title={'center': 'max1'}>,
             <Axes: title={'center': 'max2'}>],
            [<Axes: title={'center': 'max3'}>,
             <Axes: title={'center': 'max4'}>,
             <Axes: title={'center': 'max5'}>,
             <Axes: title={'center': 'max6'}>,
             <Axes: title={'center': 'max7'}>,
             <Axes: title={'center': 'max8'}>,
             <Axes: title={'center': 'max9'}>],
            [<Axes: title={'center': 'max10'}>,
             <Axes: title={'center': 'max11'}>,
             <Axes: title={'center': 'max12'}>,
             <Axes: title={'center': 'avg1'}>,
             <Axes: title={'center': 'avg2'}>,
             <Axes: title={'center': 'avg3'}>,
             <Axes: title={'center': 'avg4'}>],
            [<Axes: title={'center': 'avg5'}>,
             <Axes: title={'center': 'avg6'}>,
             <Axes: title={'center': 'avg7'}>,
             <Axes: title={'center': 'avg8'}>,
             <Axes: title={'center': 'avg9'}>,
             <Axes: title={'center': 'avg10'}>,
             <Axes: title={'center': 'avg11'}>],
            [<Axes: title={'center': 'avg12'}>,
             <Axes: title={'center': 'rain1'}>,
             <Axes: title={'center': 'rain2'}>,
             <Axes: title={'center': 'rain3'}>,
             <Axes: title={'center': 'rain4'}>,
             <Axes: title={'center': 'rain5'}>,
             <Axes: title={'center': 'rain6'}>],
            [<Axes: title={'center': 'rain7'}>,
             <Axes: title={'center': 'rain8'}>,
             <Axes: title={'center': 'rain9'}>,
             <Axes: title={'center': 'rain10'}>,
             <Axes: title={'center': 'rain11'}>],
            ])

```

```
<Axes: title={'center': 'rain12'}>, <Axes: >]], dtype=object)
```



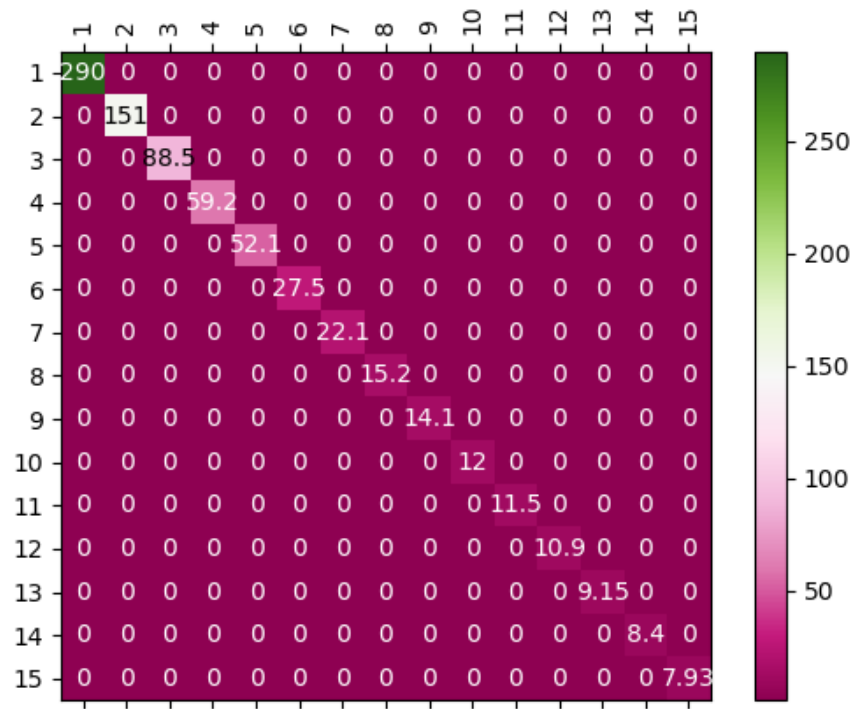
1.2 2b

```
[ ]: # Compute the SVD of the normalized climate data and store it in variables U, s, Vt. What
      # is the rank of the data?
      # YOUR PART
      U, s, Vt = np.linalg.svd(X)
      S = np.diag(s)

[ ]: rank_X = np.sum(s > 1e-10)
      print(f'Rank: {rank_X}')

Rank: 48

[ ]: # starting 15 singular values in the matrix S plot
      plot_matrix(S[:15, :15], lim=(1, S.max()), rownames = range(1,16), colnames =
      range(1,16))
```

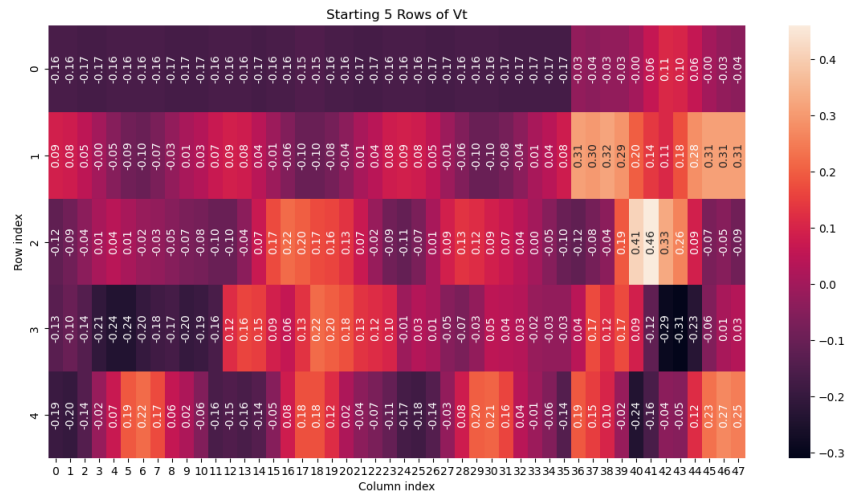


```
[ ]: # starting 5 rows of Vt

# heatmap to understand magnitude and sign of values
import seaborn as sns

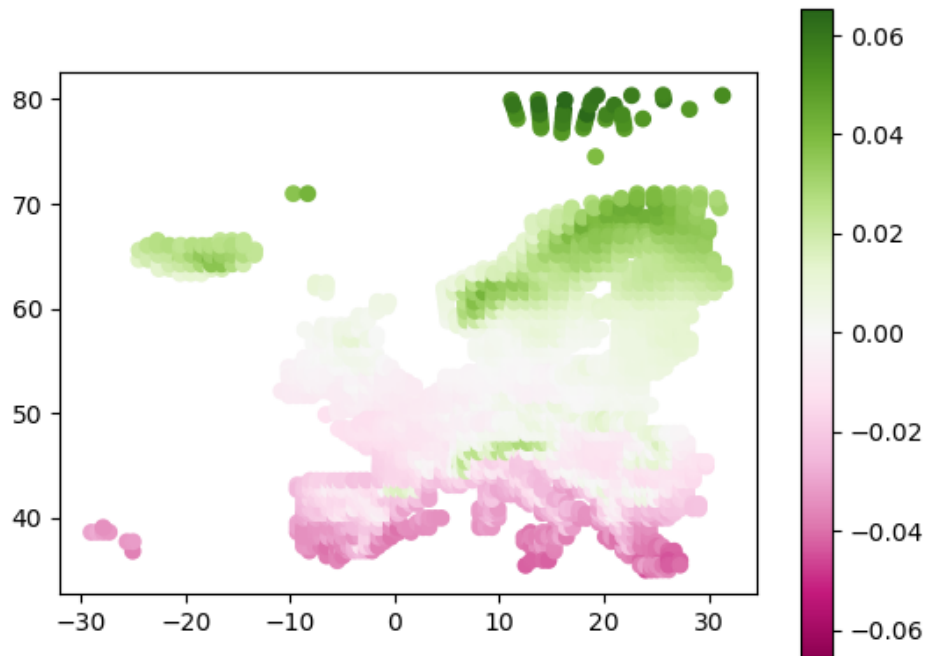
plt.figure(figsize=(14, 7))
sns.heatmap(
    np.round(Vt[:5, :], 2),
    annot=True,
    fmt=".2f",
    cbar=True,
    annot_kws={"rotation": 90}
)

plt.xlabel("Column index")
plt.ylabel("Row index")
plt.title("Starting 5 Rows of Vt")
plt.show()
```



1.3 2c

```
[ ]: # Here is an example plot.
plot_xy(lon, lat, U[:, 0])
```



```
[ ]: # For interpretation, it may also help to look at the other component matrices
      ↪ and
      # perhaps use other plot functions (e.g., plot_matrix).
      # YOUR PART

      # starting five features as subplots

fig, axs = plt.subplots(2, 3, figsize=(20, 10))
axs = axs.flatten()

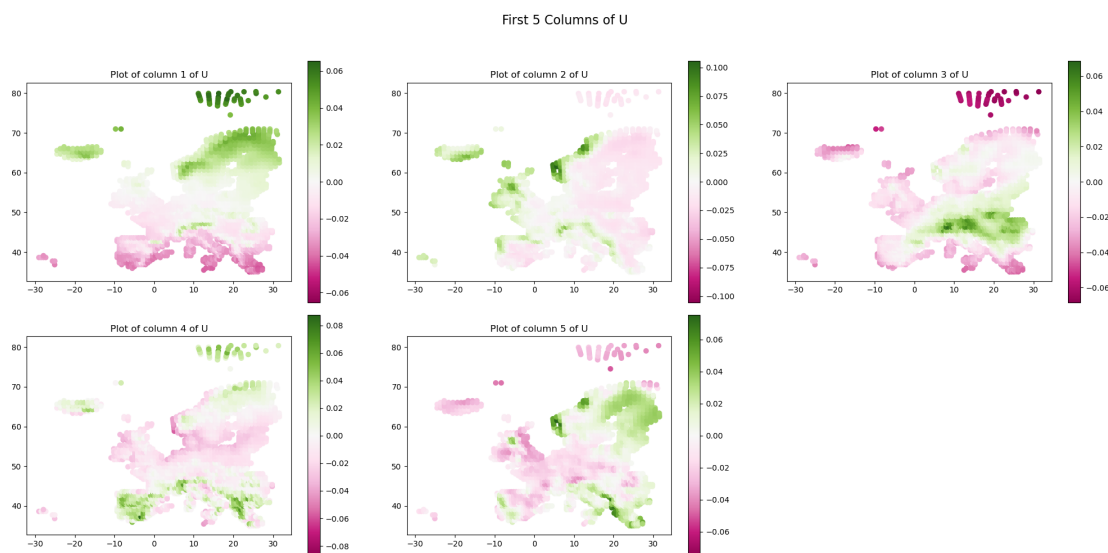
# plotting each column in a separate axis with a corresponding title, unused 6th
      ↪ axis will be removed
for i in range(5):
    plot_xy(lon, lat, U[:, i], aspect = 1, axis=axs[i]) # Use 'axis' for each
      ↪ subplot
    axs[i].set_title(f'Plot of column {i+1} of U') # Add a title to each
      ↪ subplot

fig.delaxes(axs[5])

fig.suptitle("First 5 Columns of U", fontsize=16)

# adjusting alignment to use more space
plt.tight_layout(rect=[0, 0, 1, 0.95]) # Slightly increase padding
plt.subplots_adjust(wspace=0.15, hspace=0.05) # Adjust space between subplots

plt.show()
```



```
[ ]: # nextplot
# plot_matrix(Vt[:5, :5])
np.set_printoptions(precision=2)
```

```
[ ]: # first row of Vt
print(Vt[0, :12]) #min1-12
print(Vt[0, 12:24]) #max1-12
print(Vt[0, 24:36]) #avg1-12
print(Vt[0, 36:]) #rain1-12
```

```
[-0.16 -0.16 -0.17 -0.17 -0.16 -0.16 -0.16 -0.16 -0.17 -0.17 -0.17 -0.16]
[-0.16 -0.16 -0.17 -0.16 -0.16 -0.15 -0.15 -0.16 -0.16 -0.17 -0.17 -0.16]
[-0.16 -0.16 -0.17 -0.17 -0.16 -0.16 -0.16 -0.17 -0.17 -0.17 -0.17 -0.17]
[-0.03 -0.04 -0.03 -0.03 -0.    0.06  0.11  0.1   0.06 -0.   -0.03 -0.04]
```

```
[ ]: # second row of Vt
print(Vt[1, :12]) #min1-12
print(Vt[1, 12:24]) #max1-12
print(Vt[1, 24:36]) #avg1-12
print(Vt[1, 36:]) #rain1-12
```

```
[ 0.09  0.08  0.05 -0.   -0.05 -0.09 -0.1  -0.07 -0.03  0.01  0.03  0.07]
[ 0.09  0.08  0.04 -0.01 -0.06 -0.1  -0.1  -0.08 -0.04  0.01  0.04  0.08]
[ 0.09  0.08  0.05 -0.01 -0.06 -0.1  -0.1  -0.08 -0.04  0.01  0.04  0.08]
[0.31 0.3  0.32 0.29 0.2  0.14 0.11 0.18 0.28 0.31 0.31 0.31]
```

```
[ ]: # third row of Vt
print(Vt[2, :12]) #min1-12
print(Vt[2, 12:24]) #max1-12
print(Vt[2, 24:36]) #avg1-12
print(Vt[2, 36:]) #rain1-12
```

```
[-0.12 -0.09 -0.04  0.01  0.04  0.01 -0.02 -0.03 -0.05 -0.07 -0.08 -0.1 ]
[-0.1  -0.04  0.07  0.17  0.22  0.2   0.17  0.16  0.13  0.07 -0.02 -0.09]
[-0.11 -0.07  0.01  0.09  0.13  0.12  0.09  0.07  0.04  0.   -0.05 -0.1 ]
[-0.12 -0.08 -0.04  0.19  0.41  0.46  0.33  0.26  0.09 -0.07 -0.05 -0.09]
```

```
[ ]: # fourth row of Vt
print(Vt[3, :12]) #min1-12
print(Vt[3, 12:24]) #max1-12
print(Vt[3, 24:36]) #avg1-12
print(Vt[3, 36:]) #rain1-12
```

```
[-0.13 -0.1  -0.14 -0.21 -0.24 -0.24 -0.2  -0.18 -0.17 -0.2  -0.19 -0.16]
[0.12 0.16 0.15 0.09 0.06 0.13 0.22 0.2  0.18 0.13 0.12 0.1 ]
[-0.01  0.03  0.01 -0.05 -0.07 -0.03  0.05  0.04  0.03 -0.02 -0.03 -0.03]
[ 0.04  0.17  0.12  0.17  0.09 -0.12 -0.29 -0.31 -0.23 -0.06  0.01  0.03]
```



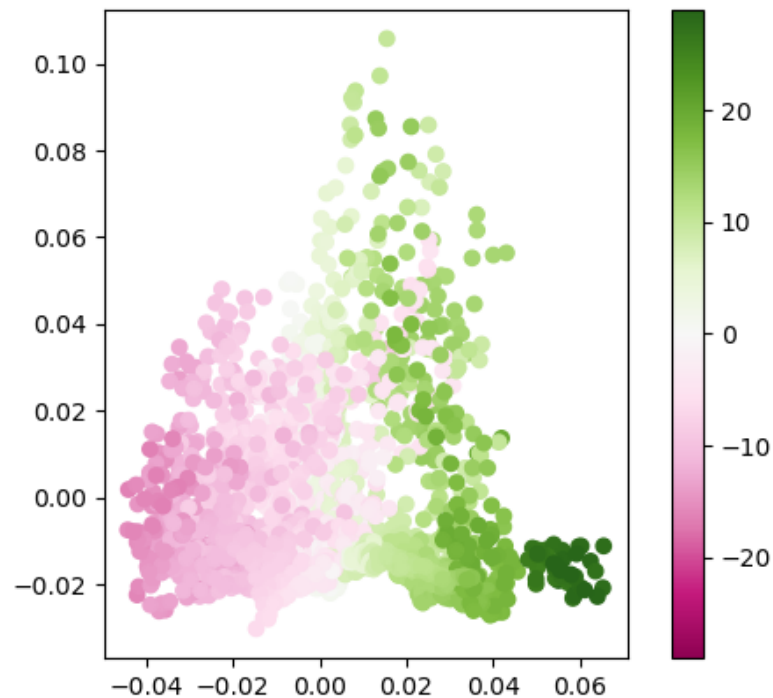
```
[ ]: # fifth row of Vt
print(Vt[4, :12]) #min1-12
print(Vt[4, 12:24]) #max1-12
print(Vt[4, 24:36]) #avg1-12
print(Vt[4, 36:]) #rain1-12

[-0.19 -0.2  -0.14 -0.02  0.07  0.19  0.22  0.17  0.06  0.02 -0.06 -0.16]
[-0.15 -0.16 -0.14 -0.05  0.08  0.18  0.18  0.12  0.02 -0.04 -0.07 -0.11]
[-0.17 -0.18 -0.14 -0.03  0.08  0.2  0.21  0.16  0.04 -0.01 -0.06 -0.14]
[ 0.19  0.15  0.1 -0.02 -0.24 -0.16 -0.04 -0.05  0.12  0.23  0.27  0.25]

[ ]: np.set_printoptions()
```

1.4 2d

```
[ ]: # Here is an example.
plot_xy(U[:, 0], U[:, 1], lat - np.mean(lat))
```



```
[ ]: # Plotting 1st column of U against each of the following for columns of U for
      ↪ North-South location
```

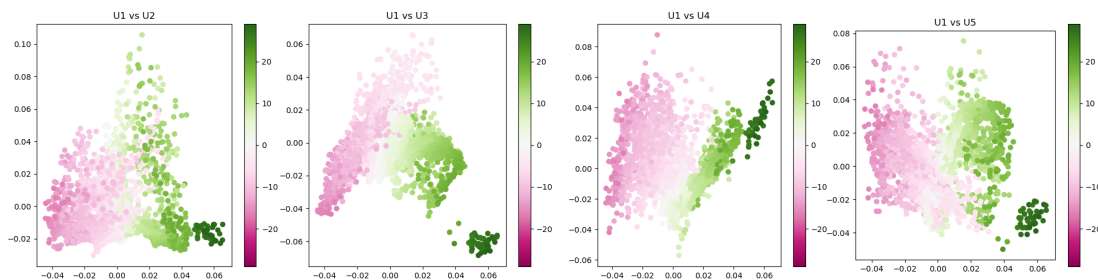
```

fig, axs = plt.subplots(1, 4, figsize=(20, 5))
axs = axs.flatten()

# plotting each pair of columns in a separate axis with a corresponding title
for i in range(1, 5):
    plot_xy(U[:, 0], U[:, i], lat - np.mean(lat), axis=axs[i-1])
    axs[i-1].set_title(f'U1 vs U{i+1}')

# visual formattings of plots
plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.subplots_adjust(wspace=0.1)
plt.show()

```



```

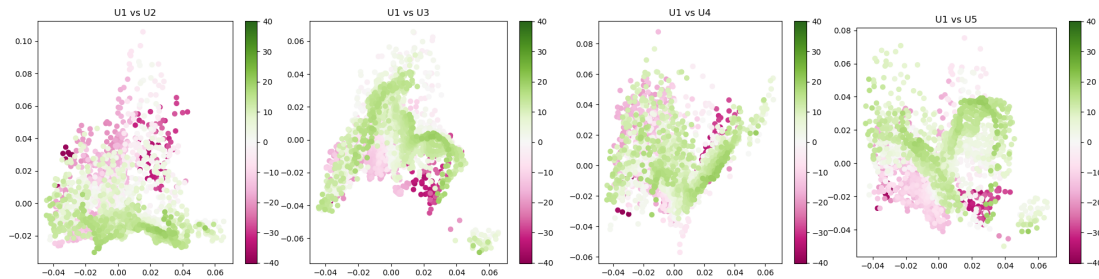
[ ]: # Next, we will plot the first column of U against each of the following for U
      ↪ columns of U for East-West location

fig, axs = plt.subplots(1, 4, figsize=(20, 5))
axs = axs.flatten()

# plotting each pair of columns in a separate axis with a corresponding title
for i in range(1, 5):
    plot_xy(U[:, 0], U[:, i], lon - np.mean(lon), axis=axs[i-1])
    axs[i-1].set_title(f'U1 vs U{i+1}')

# adjusting padding and alignment to use more space
plt.tight_layout(rect=[0, 0, 1, 0.95])
plt.subplots_adjust(wspace=0.1)
plt.show()

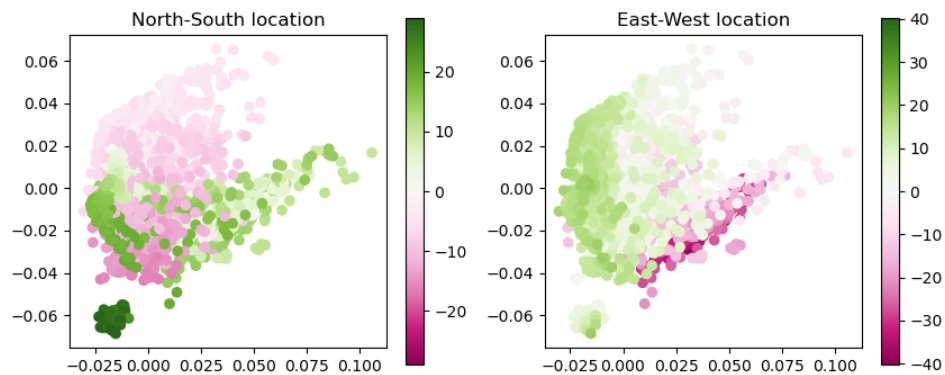
```



```
[ ]: # we will also take a look at the relation between U2 and U3
fig, axs = plt.subplots(1, 2, figsize=(10, 4))
```

```
plot_xy(U[:, 1], U[:, 2], lat - np.mean(lat), axis=axs[0])
axs[0].set_title(f'North-South location')
plot_xy(U[:, 1], U[:, 2], lon - np.mean(lon), axis=axs[1])
axs[1].set_title(f'East-West location')
```

```
[ ]: Text(0.5, 1.0, 'East-West location')
```



1.5 2e

```
[ ]: # 2e(i) Guttman-Kaiser
# YOUR PART
print(f'k selected by Guttman-Kaiser method: {np.sum(s>1)}')
```

k selected by Guttman-Kaiser method: 37

```
[ ]: # 2e(ii) 90% squared Frobenius norm
# YOUR PART#
threshold = 0.9 * (np.linalg.norm(X, 'fro') ** 2)
```

```

sum_of_squares = 0

squared_s = s**2
# aggregating sum of squares of singlar values starting from the largest until
↪ it gets equal or larger than threshold
for i in range(len(s)):
    sum_of_squares += squared_s[i]
    if sum_of_squares >= threshold:
        break
print(f'k selected by the method of 90% squared Frobenius norm: {i+1}')

```

k selected by the method of 90% squared Frobenius norm: 3

```

[ ]: # 2e(iii) Scree test

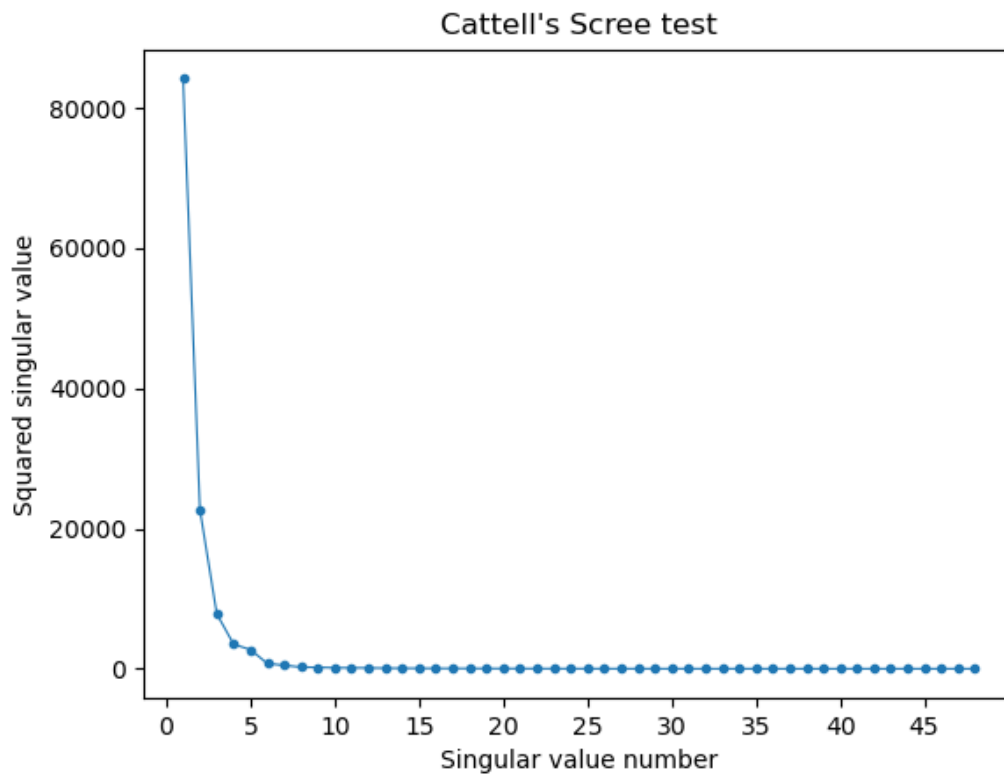
# plotting squared singular values in decreasing order (the Scree plot)
nextplot()
plt.plot(np.arange(1, len(s)+1), squared_s, marker='.', linestyle='-',
↪ linewidth = 0.8)

# label
plt.xlabel('Singular value number')
plt.ylabel('Squared singular value')
xticks = np.arange(0, len(s)+1, 5)
plt.xticks(xticks)

plt.title('Cattell\'s Scree test')
plt.show()

print(f'By visual examination of the Scree plot, we choose k equal to 6')

```



By visual examination of the Scree plot, we choose k equal to 6

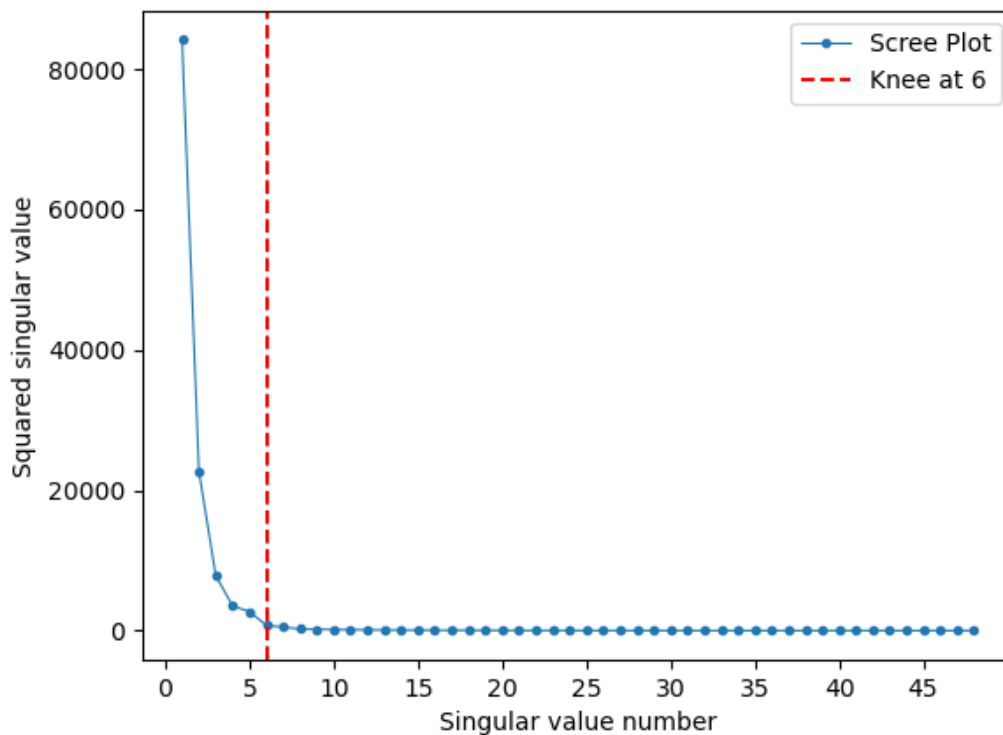
```
[ ]: # to check our choice of k=6 from visual examination of the scree plot, we will
      ↪ use a
      # Knee Locator algorithm (https://pypi.org/project/kneed/) to identify the
      ↪ point of maximum curvature
      # %pip install kneed

      from kneed import KneeLocator

      # applying KneeLocator to find the "elbow"
      knee = KneeLocator(
          range(1, len(squared_s) + 1),
          squared_s,
          curve="convex",
          direction="decreasing"
      )
      # plotting the Scree plot with the detected knee point
      nextplot()
```

```
plt.plot(range(1, len(squared_s) + 1), squared_s, marker='.', linewidth = 0.8,
        label="Scree Plot")
plt.axvline(x=knee.knee, color = 'r', linestyle='--', label=f'Knee at {knee.
        knee}')
xticks = np.arange(0, len(s)+1, 5)
plt.xticks(xticks)
plt.xlabel('Singular value number')
plt.ylabel('Squared singular value')
plt.legend()
plt.show()

print(f"Optimal number of singular values (knee): {knee.knee}")
```



Optimal number of singular values (knee): 6

```
[ ]: # 2e(iv) entropy
      # YOUR PART
      f = squared_s / np.sum(squared_s)
      entropy = - (1/np.log(min(S.shape[0], S.shape[1]))) * np.sum(f * np.log(f))
      print (f'Entropy: {entropy}')
```

```

# searching for the smallest k such that the sum of relative contributions up to
↳ k-th singular value
# is equal or larger than the entropy
sum_of_f = 0
for i in range(len(f)):
    sum_of_f += f[i]
    if sum_of_f >= entropy:
        break
print(f'k selected by entropy-based method: {i+1}')

```

Entropy: 0.2752163447341982

k selected by entropy-based method: 1

```

[ ]: # 2e(v) random flips
# Random sign matrix: np.random.choice([-1,1], X.shape)
# YOUR PART
np.random.seed(42)

# checking different values of k
changes = []

for k in range(1, min(X.shape[0], X.shape[1])+1):
    # defining the residual matrix for given k
    X_k = svdcomp(X, range(k))
    X_minus_k = X - X_k

    # constructing X_tilda_k from the residual matrix by flipping signs
    signs = np.random.choice([-1, 1], size = X_minus_k.shape)
    X_tilda_minus_k = X_minus_k * signs

    # computing the relative difference
    change = (np.linalg.norm(X_minus_k, ord=2) - np.linalg.
↳ norm(X_tilda_minus_k, ord=2)) / np.linalg.norm(X_minus_k, ord='fro')

    print(f"k={k}, difference = {change:.6f}")

    changes.append(change)

```

```

k=1, difference =0.478194
k=2, difference =0.351524
k=3, difference =0.395349
k=4, difference =0.461332
k=5, difference =0.236031
k=6, difference =0.194869
k=7, difference =0.111044
k=8, difference =0.169182
k=9, difference =0.119629
k=10, difference =0.151240

```

```

k=11, difference =0.171992
k=12, difference =0.137750
k=13, difference =0.114050
k=14, difference =0.141650
k=15, difference =0.140642
k=16, difference =0.169491
k=17, difference =0.173571
k=18, difference =0.151970
k=19, difference =0.149593
k=20, difference =0.200942
k=21, difference =0.175650
k=22, difference =0.166248
k=23, difference =0.150726
k=24, difference =0.152478
k=25, difference =0.175884
k=26, difference =0.139230
k=27, difference =0.172028
k=28, difference =0.188804
k=29, difference =0.180012
k=30, difference =0.173909
k=31, difference =0.197062
k=32, difference =0.253494
k=33, difference =0.252325
k=34, difference =0.233611
k=35, difference =0.221058
k=36, difference =0.256456
k=37, difference =0.299118
k=38, difference =0.275386
k=39, difference =0.230702
k=40, difference =0.202277
k=41, difference =0.200174
k=42, difference =0.199246
k=43, difference =0.236249
k=44, difference =0.239352
k=45, difference =0.300221
k=46, difference =0.409595
k=47, difference =0.453833
k=48, difference =0.546510

```

```

[ ]: # we will plot the relative change in spectral norms as the function of k and
      ↪ choose the k with the minimal change value
plt.figure(figsize=(5, 5))
plt.plot(np.arange(1, len(s)+1), changes, marker='.', linestyle='-', linewidth=
      ↪ 0.8, label="k")

# labeling and adding ticks to the axes
plt.xlabel('k')

```



```

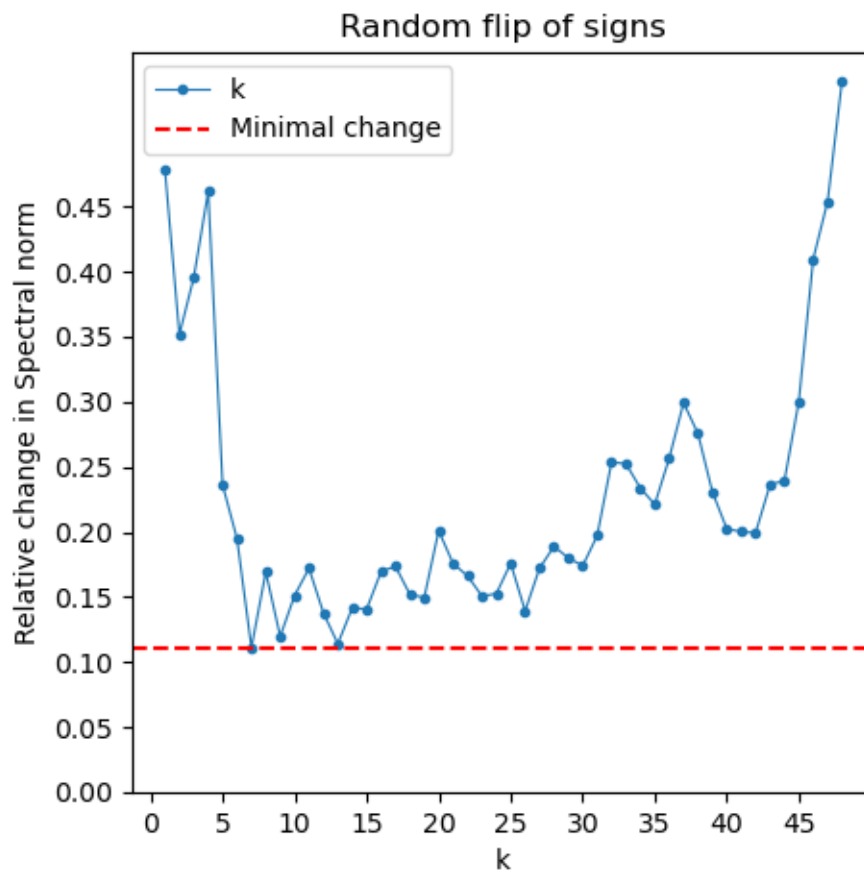
plt.ylabel('Relative change in Spectral norm')
xticks = np.arange(0, len(s)+1, 5)
yticks = np.arange(0, 0.5, 0.05)
plt.xticks(xticks)
plt.yticks(yticks)

# showing the minimal value
plt.axhline(y=min(changes), color='r', linestyle='--', label='Minimal change')

plt.legend()
plt.title('Random flip of signs')
plt.show()

print(f'Minimal change at {min(changes)} with k={np.argmin(changes)+1}')

```



Minimal change at 0.1110435279724272 with k=7

1.6 2f

```
[ ]: # defining the ranges of values for k and epsilon
k_range = [1, 2, 5, 10, 48]
epsilon_range = np.arange(0, 2.1, 0.1)

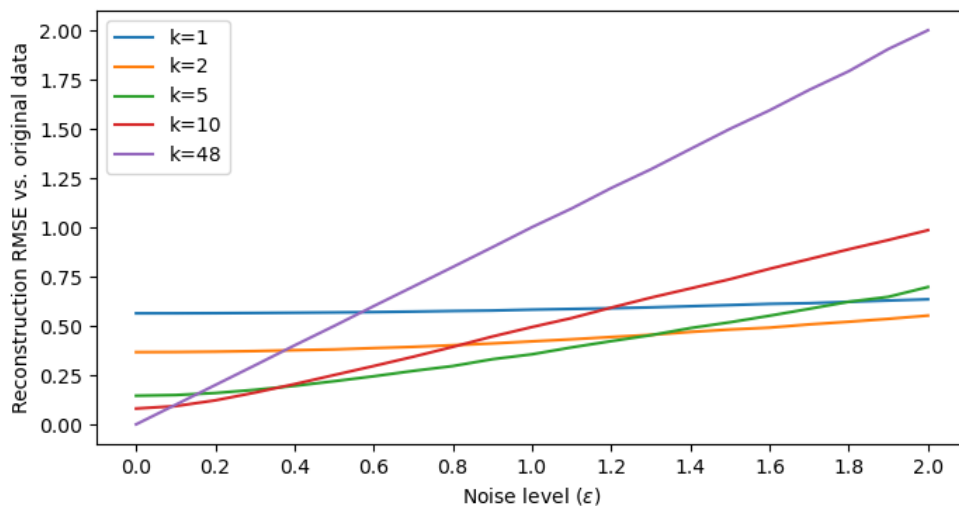
RMSE_results = {}

# computing RMSE for different values of k and epsilon
for k in k_range:
    RMSE_k = []
    for e in epsilon_range:
        X_noise = X + np.random.randn(*X.shape) * e
        X_approx = svdcomp(X_noise, range(k))
        RMSE = 1/(np.sqrt(X.shape[0] * X.shape[1])) * np.linalg.norm(X -
↪X_approx, ord='fro')
        RMSE_k.append(RMSE)
    RMSE_results[k] = RMSE_k

[ ]: # Here is the empty plot that you need to fill (one line per choice of Z: RSME
↪between
# original X and the reconstruction from size-Z SVD of noisy versions)
# YOUR PART
plt.figure(figsize=(8, 4))
for k, rmse_values in RMSE_results.items():
    plt.plot(epsilon_range, rmse_values, label=f'k={k}')

# adding labels, title and legend
plt.xlabel(r"Noise level ($\epsilon$)")
xticks = np.arange(0, 2.1, 0.2)
plt.xticks(xticks)
plt.ylabel("Reconstruction RMSE vs. original data")
plt.legend()

plt.show()
```



```
[ ]: X_approx = svdcomp(X, range(1))
RMSE = 1/(np.sqrt(X.shape[0] * X.shape[1])) * np.linalg.norm(X - X_approx,
ord='fro')
RMSE
```

```
[ ]: 0.5642797271156014
```

a03_3

November 14, 2025

1 3 SVD and Clustering

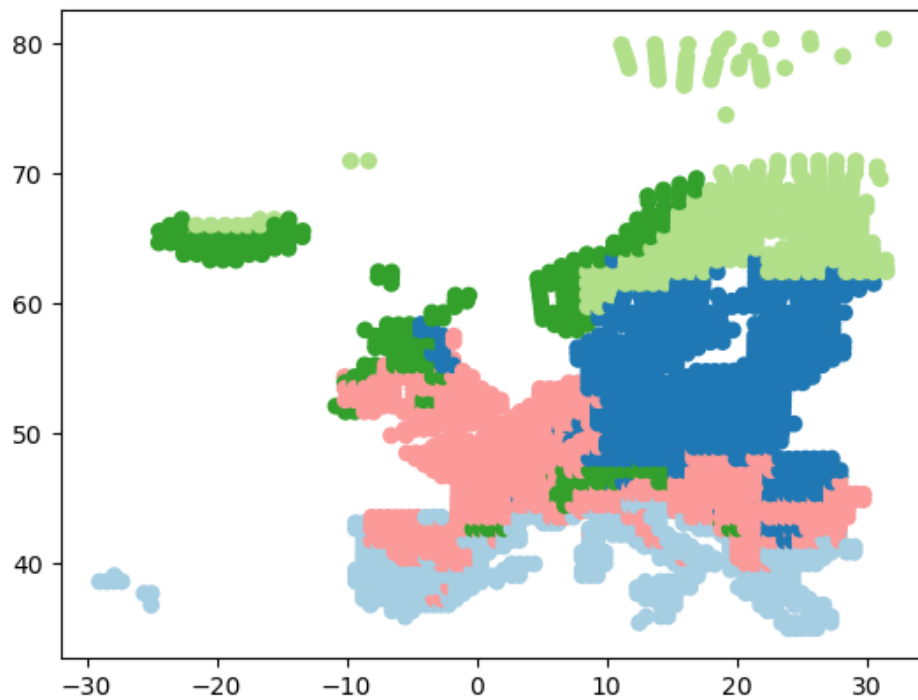
Report of Jonas Ortner: 2265527 Marmee Pandya: 1963521

```
[1]: import matplotlib.pyplot as plt
      from sklearn.cluster import KMeans

      %load_ext autoreload
      %autoreload 2
      from a03_helper import *
```

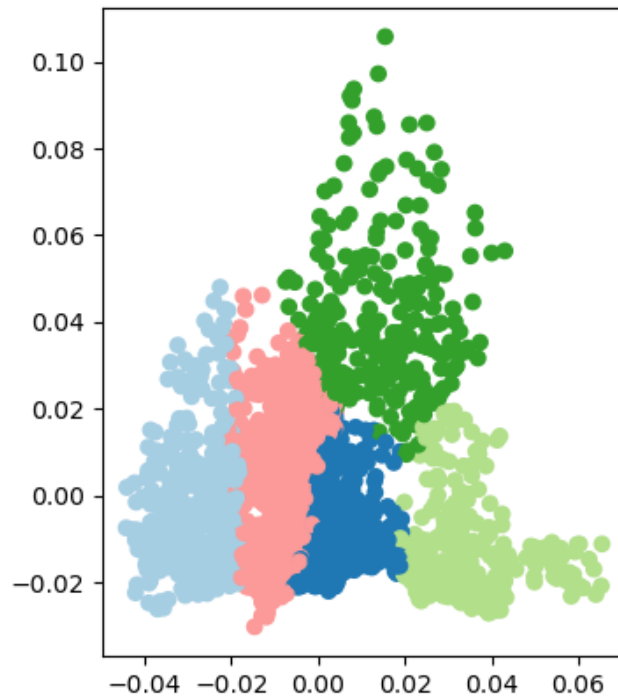
```
[2]: # Cluster the normalized climate data into 5 clusters using k-means and store
      # the vector giving the cluster labels for each location.
      climate = pd.read_csv("data/worldclim.csv")
      X = (climate - climate.mean()) / climate.std()
      X_clusters = KMeans(5).fit(X).labels_
```

```
[17]: # Plot the results to the map: use the cluster labels to give the color to each
      # point.
      plot_xy(lon, lat, X_clusters)
      plt.savefig('images/a03_3_clusters_by_coordinates.png', dpi=300,
                  bbox_inches='tight')
```



1.1 3b

```
[18]: U, s, Vt = np.linalg.svd(X)
      plot_xy(U[:, 0], U[:, 1], X_clusters)
      plt.savefig('images/a03_3_clusters_first_two_principal_components.png',
                  dpi=300, bbox_inches='tight')
```

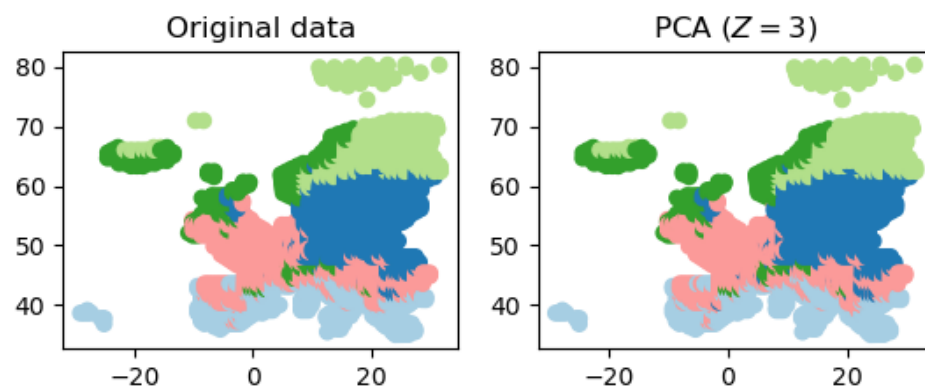


1.2 3c

```
[15]: # Compute the PCA scores, store in Z (of shape N x k)
Z = 3
U, s, Vt = np.linalg.svd(X)
S = np.diag(s)
# YOUR PART HERE
#computing score matrix
S_PCA = U[:, :Z] * np.diag(S)[:Z]

[16]: # cluster and visualize
S_PCA_clusters = KMeans(5).fit(S_PCA).labels_
# match clusters as well as possible (try without)
S_PCA_clusters = match_categories(X_clusters, S_PCA_clusters)
nextplot()
axs = plt.gcf().subplots(1, 2)
plot_xy(lon, lat, X_clusters, axis=axs[0])
axs[0].set_title("Original data")
plot_xy(lon, lat, S_PCA_clusters, axis=axs[1])
axs[1].set_title(f"PCA $(Z={Z})$")
```

```
plt.savefig(f'images/a03_3_clusters_pca_comparison_z{Z}.png', dpi=300,
            ↳bbox_inches='tight')
```



[]: